



Architecture and Software Product Lines

Coming together is a beginning. Keeping together is progress. Working together is success.

—Henry Ford

A software architecture represents a significant investment of time and effort, usually by senior talent. So it is natural to want to maximize the return on this investment by reusing an architecture across multiple systems.

There are many ways this happens in practice. The patterns we discussed in Chapter 13 are a big step in this direction; using a pattern is reusing a package of architectural decisions (albeit not a complete architecture). And strictly speaking, every time you make a change to a system, you are reusing its architecture (or whatever portion of its architecture you don't have to change).

This chapter shows yet another way to reuse a software architecture (and many other assets as well) across a family of related systems, and the benefits that doing so can bring. Many software-producing organizations tend to produce systems or products that resemble each other more than they differ. This is an opportunity for reusing the architecture across these similar products. These software product lines simplify the creation of new members of a family of similar systems.

This kind of reuse has been shown to bring substantial benefits that include reduced cost of construction, higher quality, and greatly reduced time to market. This is the lure of the software product line approach to system building.

The Software Engineering Institute defines a software product line as “a set of software-intensive systems sharing a common, managed set of features that satisfy the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way.”

The vision is of a set of reusable assets (called core assets) based on a common architecture and the software elements that populate that architecture. The

core assets also include designs and their documentation, user manuals, project management artifacts such as budgets and schedules, software test plans and test cases, and more.

The product line approach works because the core assets were built specifically to support multiple members of the same family of products. Hence, reusing them is faster and less expensive than reinventing those software assets for each new product or system in the organization's portfolio. Core assets, including the architecture, are usually designed with built-in variation points—places where they can be quickly tailored in preplanned ways.

Once the core assets are in place, system building becomes a matter of

- Accessing the appropriate assets in the core asset base
- Exercising the variation points to configure them as required for the system being built
- Assembling that system

In the ideal case, this can be done automatically. Additional software developed for an individual product, if needed at all, tends to account for a small fraction of the total software. Integration and testing replace design and coding as the predominant activities.

Product lines are nothing new in manufacturing. Many historians trace the concept to Eli Whitney's use of interchangeable parts to build rifles in the early 1800s, but earlier examples also exist. Today, there are hundreds of examples in manufacturing: think of the products of companies like General Motors, Toyota, Boeing, Airbus, Dell, even McDonald's, and the portfolio of similar products that each one produces. Each company exploits commonality in different ways. Boeing, for example, developed the 757 and 767 in tandem, and the parts lists of these two very different aircraft overlap by about 60 percent.

The improvements in cost, time to market, and productivity that come with a successful software product line can be breathtaking. Consider:

- Nokia credits the software product line approach with giving it flexibility to bring over a dozen phones to market each year, as opposed to the three or so it could manage before, all with an unprecedented variety of features.
- Cummins, Inc., was able to reduce the time it takes to produce the software for a diesel engine from about a year to about a week.
- Hewlett-Packard builds products using one-quarter of the staff, in one-third of the time, and with one twenty-fifth the number of defects, compared with software built before the advent of software product line engineering.
- Deutsche Bank estimates \$4 million in savings per year realized from building its global transaction and settlement software as a product line.
- Philips reports reduced faults during integration in its high-end television portfolio by adopting the product line approach. Product diversity used to be one of the top three concerns of their architects. Now it doesn't even make the list of concerns at all; the product line approach has taken

software development off the critical path—the software no longer determines the delivery date of the product.

- With a product line of satellite ground control systems it commissioned, the U.S. National Reconnaissance Office reported the first product requiring 10 percent the expected number of developers and having one-tenth the expected number of defects.
- In Philips's medical systems product line, the software product line approach has cut both software defects and time to market by more than half.

Creating a successful product line depends on a coordinated strategy involving software engineering, technical management, and organization management. Because this is a book on software architecture, we focus on the architectural aspects of software product lines, but all aspects must work together in order for an organization to successfully create a product line.

That Silver Lining Might Have a Cloud

The software product line paradigm is a powerful way to leverage an investment in architecture (and other core assets) into a family of related systems and thus see order-of-magnitude improvements in time to market, quality, and productivity. These results are possible and have been demonstrated by companies large and small in many different domains. The effects are real. Further, data from many sources and companies confirms with astonishing consistency that, to make the investment pay off, an organization needs to build only three products. This is the minimum number we would expect to have in a product line.

But other results are possible as well, and a spectacular crash-and-burn is not out of the question when trying to adopt this approach. Product line practice, like any technology, needs careful thought given to its adoption, and a company's history, situation, and culture must be taken into account. Factors that can contribute to product line failure include these:

- Lack of a champion in a position of sufficient control and visibility
- Failure of management to provide sustained and unwavering support
- Reluctance of middle managers to relinquish autocratic control of projects
- Failure to clearly identify business goals for adopting the product line approach
- Abandoning the approach at the first sign of difficulty
- Failure to adequately train staff in the approach and failure to explain or justify the change adequately
- Lack of discipline in managing the architecture's variation points
- Scoping the product line too broadly or too narrowly
- Lack of product line tooling to help manage and exercise the variation points

Fortunately, there are strategies for overcoming most of these factors. One good strategy is to launch a small but visible pilot project to demonstrate the quantitative benefits of software product lines. The pilot can be staffed by those most willing to try something new while the skeptics go about their business. It can work out process issues, clarify roles and responsibilities, and in general work out the bugs before the approach is transitioned to a wider setting.

—PCC

25.1 An Example of Product Line Variability

The following example will help us illustrate the concept of product line variability. In a product line of software to support U.S. bank loan offices, suppose we have a software module that calculates what a customer owes in the current month. For 18 of the 21 products in our product line, this module is completely adequate. However, our company is about to enter the market in the state of Delaware, which has certain laws that affect what a customer can owe. For the three products we plan to sell in Delaware, we need a module that differs from the “standard” module. Analysis shows that the difference will affect about 250 lines of source code in our 8,000-line module.

To build one of the Delaware products, what do we do? An obvious option is to copy the module, change the 250 or so lines, and use the new version in the three products. This practice is called “clone-and-own”—the new projects “clone” the module, change it, and then “own” the new version. Most companies, when faced with this situation, resort to clone-and-own. It’s expedient in that it provides a quick start to a new product, but it comes with a substantial cost down the road.

The problem with clone-and-own is that it doesn’t scale. Suppose each of our 21 products comprises roughly 100 modules. If each module is allowed to diverge for each product, that’s potentially 2,100 modules that the maintenance staff has to deal with, each one spiraling off on its own separate maintenance trajectory based on the needs of the lone project each version is used in. Many companies’ growth in a market is limited—brought to a halt, in fact—by their inability to staff the maintenance of so many separate versions of so many different assets composing the products in their portfolio. An organization fielding several versions of several products finds itself dealing with a staggeringly complex code base. The strain begins to show when a systematic change needs to be made to all of the products—for example, to add a new feature, or migrate to a new platform, or make the user interface work in a different language. Because each version of each component used in each product has been allowed to evolve separately, now suddenly making a systematic change becomes prohibitively expensive (and

only gets worse each time a new product is added—the labor involved grows as the *square* of the number of products). It only takes a few such portfolio-wide changes before organizations feel that they’ve hit a wall of complexity and expense.

So much for clone-and-own. What else can we do? Instead of allowing up to 21 versions of each module, we would much rather find a way to take advantage of the fact that these nearly identical modules vary only in small, well-defined ways. To take advantage of their similarities, we introduce a *variation mechanism* into the module. (Variation mechanisms are often realized as tactics, such as the “defer binding” set of tactics described in Chapter 7.) This variation mechanism will let us maintain a single module that can adapt to the range of variations in the applications (in our example, the 21 banking products) that it has to support. If we plan to market our products in states that, like Delaware, have their own laws affecting what a customer owes, we may need to support additional variations of the module. So our variation mechanism should be able to accommodate those possibilities as well.

The payoff for this up-front planning is that an asset used in any of the products exists as a *single* version that (through the exercising of built-in variation mechanisms) works for all of the products in the product line. And now, making a portfolio-wide change merely consists of changing the core assets that are affected. Because all future versions of all products use the same core assets, changing the core asset base has the effect of changing all of the products in the organization’s portfolio.

25.2 What Makes a Software Product Line Work?

What makes product lines succeed is that the commonalities shared by the products can be exploited through reuse to achieve production economies. The potential for reuse is broad and far-ranging, including the following:

- *Requirements.* Most of the requirements are common with those of earlier systems and so can be reused. In fact, many organizations simply maintain a single set of requirements that apply across the entire family as a core asset; the requirements for a particular system are then written as “delta” documents off the full set. In any case, most of the effort consumed by requirements analysis is saved from system to system.
- *Architectural design.* An architecture for a software system represents a large investment of time from the organization’s most talented engineers. As we have seen, the quality goals for a system—performance, reliability, modifiability, and so forth—are largely promoted or inhibited once the architecture is in place. If the architecture is wrong, the system cannot be

saved. For a new product, however, this most important design step is already done and need not be repeated.

- *Software elements.* Software elements are applicable across individual products. Element reuse includes the (often difficult) initial design work. Design successes are captured and reused; design dead ends are avoided, not repeated. This includes design of each element's interface, its documentation, its test plans and procedures, and any models (such as performance models) used to predict or measure its behavior. One reusable set of elements is the system's user interface, which represents an enormous and vital set of design decisions. And as a result of this interface reuse, products in a product line usually enjoy the same look and feel as each other, an advantage in the marketplace.
- *Modeling and analysis.* Performance models, schedulability analysis, distributed system issues (such as proving the absence of deadlock), allocation of processes to processors, fault tolerance schemes, and network load policies all carry over from product to product. Companies that build real-time distributed systems report that one of the major headaches associated with production has all but vanished. When they field a new product in their product line, they have high confidence that the timing problems have been worked out and that the bugs associated with distributed computing—synchronization, network loading, and absence of deadlock—have been eliminated.
- *Testing.* Test plans, test processes, test cases, test data, test harnesses, and the communication paths required to report and fix problems are already in place.
- *Project planning artifacts.* Budgeting and scheduling are more predictable because experience is a high-fidelity indicator of future performance. Work breakdown structures need not be invented each time. Teams, team size, and team composition are all easily determined.

All of these represent valuable core assets, each of which can be imbued with its own variation points that can be exercised to build a product. We'll look at architectural variation points later in this chapter, but for now imagine that any artifact represented by text can consist of text blocks that are exposed or hidden for a particular product. Thus, the artifact that is maintained in the core asset base represents a superset of any version that will be produced for a product.

Artifact reuse in turn enables reuse of knowledge:

- *Processes, methods, and tools.* Configuration control procedures and facilities, documentation plans and approval processes, tool environments, system generation and distribution procedures, coding standards, and many other day-to-day engineering support activities can all be carried over from product to product. The software development process is in place and has been used before.

Giving Software Reuse a New Lease on Life

Software product lines rely on reuse, but reuse has a long but less than stellar history in software engineering, with the promise almost always exceeding the payoff. One reason for this failure is that until now reuse has been predicated on the idea of “If you build it, they will come.” A reuse library is stocked with snippets from previous projects, and developers are expected to check it first before coding new elements. Almost everything conspires against this model. If the library is too sparse, the developer will not find anything of use and will stop looking. If the library is too rich, it will be hard to understand and search. If the elements are too small, it is easier to rewrite them than to find them and carry out whatever modifications they might need. If the elements are too large, it is difficult to determine exactly what they do in detail, which in any case is not likely to be exactly right for the new application. In most reuse libraries, pedigree is hazy at best. The developer cannot be sure exactly what the element does, how reliable it is, or under what conditions it was tested. And there is almost never a match between the quality attributes needed for the new application and those provided by the elements in the library.

In any case, it is common that the elements were written for a different architectural model than the one the developer of the new system is using. Even if you find something that does the right thing with the right quality attributes, it is doubtful that it will be the right kind of architectural element (if you need an object, you might find a process), that it will have the right interaction protocol, that it will comply with the new application’s error-handling or failover policies, and so on.

This has led to so many reuse failures that many project managers have given up on the idea. “Bah!” they exclaim. “We tried reuse before, and it doesn’t work!”

Software product lines make reuse work by establishing a strict context for it. The architecture is defined; the functionality is set; the quality attributes are known. Nothing is placed in the reuse library—or “core asset base” in product line terms—that was not built to be reused in that product line. Product lines work by relying on strategic or planned, not opportunistic, reuse.

—PCC

- *People.* Because of the commonality of applications, personnel can be transferred among projects as required. Their expertise is applicable across the entire line.
- *Exemplar systems.* Deployed products serve as high-quality demonstration prototypes or engineering models of performance, security, safety, and reliability.

- *Defect elimination.* Product lines enhance quality because each new system takes advantage of the defect elimination in its forebears. Developer and customer confidence both rise with each new instantiation. The more complicated the system, the higher the payoff for solving vexing performance, distribution, reliability, and other engineering issues once for the entire family.

All of this reuse helps products launch more quickly, with higher quality, lower cost, and more predictable budget and schedule. This is critical for getting a product to market in a timely fashion. However, these benefits do not come for free. A product line may require a substantial up-front investment of time and effort to set up and manage, as well as to keep the core assets responsive to changing market needs.

25.3 Product Line Scope

One of the most important inputs to an architect building an architecture for a software product line is the product line's scope. A product line's scope is a statement about what systems an organization is willing to build as part of its line and what systems it is not willing to build. Defining a product line's scope is like drawing a doughnut in the space of all possible systems, as shown in Figure 25.1. The doughnut's center represents the systems that the organization could easily build using its base of core assets; these are within its production capability. Systems outside the doughnut are out of scope because they are ones the product line's core assets are not well equipped to handle; this would be like asking Toyota to build, say, apple pies on one of its automotive assembly lines.

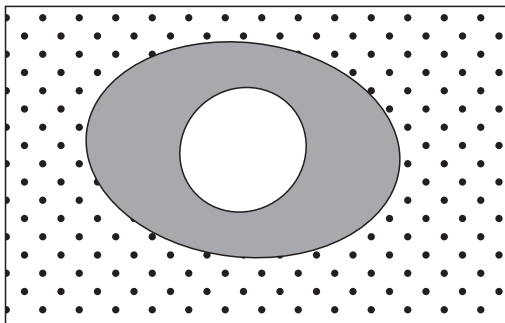


FIGURE 25.1 The space of all possible systems is divided into areas within scope (white), areas outside of scope (speckled), and areas that require case-by-case disposition (gray).

Systems on the doughnut itself could be handled, but with some effort. These often represent invitations from the marketplace asking the organization to extend its product line. To take advantage of such an opportunity, the organization would have to broaden its production capability—that is, make its core asset base able to handle the new product. These opportunities require case-by-case disposition as they arise, to see if the potential payoff (such as entry into a slightly different area of the market) would outweigh the cost to modify the core assets. This would be like asking Toyota to build a riding lawnmower.

The scope represents the organization's best prediction about what products it will be asked to build in the foreseeable future. Input to the scoping process comes from the organization's strategic planners, marketing staff, domain analysts who can catalog similar systems (both existing and on the drawing board), and technology experts.

A product line scope is a critical factor in the success of the product line. Scope too narrowly (the products only vary in a small number of features) and an insufficient number of products will be derived to justify the investment in development. Scope too broadly (the products vary in kind as well as in features) and the effort required to develop individual products from the core assets is too great to lead to significant savings. Scope can be refined as a portion of the initial establishment of the product line or opportunistically depending on the product line adoption strategy (see the section on adoption strategies in Section 25.8).

The problem in defining the scope is not in finding commonality—a creative architect can find points of commonality between *any* two systems—but in finding commonality that can be exploited to substantially reduce the cost of constructing the systems that an organization intends to build. When considering scope, more than just the systems being built should be considered. Market segmentation and types of customer interactions assumed will help determine the scope of any particular product line. For example, Philips, the Dutch manufacturer of consumer electronics, has distinct product lines for home video electronic systems and digital video communication. Video is the common thread, but one is a mass market, where the customer is assumed to have very little video sophistication, and the other is a much smaller market consisting purely of video professionals. The products being developed reflect these assumptions about the sophistication of customers and the amount of care each customer will receive. These differences were sufficient to keep Philips from attempting to develop a single product line for both markets.

Narrowly scoped product lines offer opportunities to build specialized tools to support the specification of new products. For example, General Motors' Powertrain division builds a software product line of automotive software. It makes an individual product from its product line core assets based on contracts stored in a database. Each element has well-defined interfaces and possible variation points. A tool searches the database based on desired features and assembles the product.

The scope definition is vital to the product line architect because the scope defines what is common across all members of the product line, and the specific ways in which the products differ from each other. The fixed part of a product line architecture reflects what is constant, and the architecture's variation points accommodate the variations among products.

25.4 The Quality Attribute of Variability

Scoping decisions, which tell the product line architect what kinds of systems are “in” and what kinds of systems are “out” of the product line, lead to the introduction of variability in the core assets. In fact, the quality attribute of variability is most closely associated with product lines. Some may feature high-performance products, or high-security products, or high-availability products, but *all* product lines feature variability aimed at satisfying the commonalities and variations identified by the product line's scope.

We introduced variability in Chapter 12. There we said that variability is a special form of modifiability, pertaining to the ability of a core asset to adapt to usages in the different product contexts that are within the product line scope. The goal of variability in a software product line is to make it easy to build and maintain products in the product line over time.

Table 25.1 gives the general scenario for variability. The source is some actor in the product line organization who identifies a need for variation; this actor is probably someone involved in setting the product line's scope, such as a marketer.

Identifying variation is a constant, iterative process in the life of a software product line. Because of the many different ways a product can vary, particular variants can be identified at virtually any time during the development process. Some variations are identified during product line requirement elicitations; others, during architecture design; and still others, during implementation. Variations may also be identified during implementation of the second (and subsequent) products as well.

Product line architectures feature *variability* as an important quality attribute. They achieve this by incorporation of *variation mechanisms*, which we will discuss in more detail shortly.

25.5 The Role of a Product Line Architecture

Of all of the assets in a core asset repository, the software architecture plays the most central role. There is both a tactical and a strategic reason for this.

TABLE 25.1 The General Scenario for Variability

Portion of Scenario	Possible Values
Source	Actor requesting variability
Stimulus	Requests to support variations in the following: ▪ Hardware ▪ Feature sets ▪ Technologies ▪ User interfaces ▪ Quality attributes ▪ ... and more for the range of products affected, such as: ▪ All ▪ A specified subset ▪ Those that include feature set x ▪ New products
Environment	Variants are to be created at: ▪ Runtime ▪ Build time ▪ Development time
Artifact	Asset(s) affected, such as: ▪ Requirements ▪ Architecture ▪ Component x ▪ Test suite y ▪ Project plan z ▪ ... and more
Response	The requested variants can be created.
Response measure	A specified cost and/or time to create the core assets and to create the variants using these core assets

The tactical reason is the importance the architecture plays in building products in a product line. The essence of building a successful software product line is discriminating between what is expected to remain constant across all family members and what is expected to vary. Software architecture is ideal for handling this variation, because all architectures are abstractions that admit multiple instances. By its very nature every architecture is a statement about what we expect to remain constant and what we admit may vary. For example, interfaces to components are designed to remain stable, with anticipated changes hidden behind those interfaces.

In a software product line, the architecture has to encompass both the varying and the nonvarying aspects. A product line architecture must be designed to accommodate a set of explicitly allowed variations. Thus, identifying

the allowable variations is part of the architect's responsibility, as is providing built-in mechanisms for achieving them. Those variations may be substantial. Products in a software product line exist simultaneously and may vary in terms of their behavior, quality attributes, platform, network, physical configuration, middleware, scale factors, and so forth.

The strategic reason has to do with the capability it imparts to an organization outside the realm of an existing product line. As we saw in Chapters 2 and 3, an architecture can serve as a technical platform for launching new applications and even new business models, and it can serve as a springboard for an organization diving into a new business area. This seems to be especially true for product line architectures. There are many cases where an organization has taken advantage of its production capability—that is, its core asset base crowned by a product line architecture—by using that capability to enter new markets. For example, Cummins took its product line of automotive diesel engines to enter and quickly dominate the neighboring market for industrial diesel engines. Industrial diesel engines power things like rock crushers and ski lifts, markets of low volume and high specialization. Systems in that market built uniquely for each application are expensive and don't yield a high return. But a product line that includes industrial diesel engines in its scope, and whose production capability supports industrial diesel engines, is a recipe for rapid market capture.

A product line architect needs to consider three things that are unique to product line architectures:

- *Identifying variation points.* This is done by using the scope definition and product line requirements as input. The product line architect determines where in the architecture variation points should be made available to support the rapid building of products.
- *Supporting variation points.* This is done by introducing variation mechanisms, which will be discussed in the next section.
- *Evaluating the architecture for product line suitability,* which will be discussed later in this chapter.

25.6 Variation Mechanisms

In a conventional architecture, the mechanism for achieving different instances often comes down to modifying the code. But in a software product line, modifying code is undesirable, because this leads to a large number of separately maintained implementations that quickly outstrip an organization's ability to keep them up to date and consistent.

Three primary *architectural* variation mechanisms are these:

- *Inclusion or omission of elements.* This decision can be reflected in the build procedures for different products, or the implementation of an element can be conditionally compiled based on some parameter indicating its presence or absence.
- *Inclusion of a different number of replicated elements.* For instance, high-capacity variants might be produced by adding more servers—the actual number should be unspecified, as a point of variation, and may be done dynamically.
- *Selection of different versions of elements that have the same interface but different behavioral or quality attribute characteristics.* Selection can occur at compile time, build time, or runtime. Selection mechanisms include static libraries, which contain external functions linked after compilation time; dynamic link libraries, which have the flexibility of static libraries but defer the decision until runtime based on context and execution conditions; and add-ons (e.g., plug-ins, extensions, and themes), which add or modify application functionality at runtime. By changing the libraries, we can change the implementation of functions whose names and signatures are known.

Some variation mechanisms can be introduced that change aspects of a particular software element. Modifying the source code each time the element is used in a new product—that is, clone-and-own—falls into this category, although it is undesirable. More sophisticated techniques include the following:

- *Extension points.* These are identified places in the architecture where additional behavior or functionality can be safely added.
- *Reflection.* This is the ability of a program to manipulate data on itself or its execution environment or state. Reflective programs can adjust their behavior based on their context.
- *Overloading.* This is a means of reusing a named functionality to operate on different types. Overloading promotes code reuse, but at the cost of understandability and code complexity.

Other commonly used variation mechanisms include those in Table 25.2.

Choosing the right variation mechanism affects numerous costs:

- The skill set required to implement, or learn and use, the specific variation mechanism, such as server or framework programming
- The one-time costs of building or acquiring the tools (such as compilers or generators) required to create the variation mechanism
- The recurring cost and time to exercise the variation mechanism

The choice of variation mechanism also affects downstream users and developers:

- The targeted group of users that use the mechanism for product-specific adaptation, such as product developer, integrator, system administrator, and end user

Finally, the choice of variation mechanism affects product quality:

- The impact of the variation mechanism on quality, such as possible performance penalties or memory consumption
- The impact on the mechanism's maintainability

The architect should document the choice of variation mechanisms. In fact, the documentation of variation mechanisms is the primary way in which the documentation for a product line architecture differs from that of a conventional architecture. In the documentation template we presented in Chapter 18, the section

TABLE 25.2 Common Variation Mechanisms

Variation Mechanism	Properties Relevant to Building the Core Assets	Properties Relevant to Exercising the Variation Mechanism When Building Products
Inheritance; specializing or generalizing a particular class	Cost: Medium Skills: Object-oriented languages	Stakeholder: Product developers Tools: Compiler Cost: Medium
Component substitution	Cost: Medium Skills: Interface definitions	Stakeholder: Product developer, system administrator Tools: Compiler Cost: Low
Add-ons, plug-ins	Cost: High Skills: Framework programming	Stakeholder: End user Tools: None Cost: Low
Templates	Cost: Medium Skills: Abstractions	Stakeholder: Product developer, system administrator Tools: None Cost: Medium
Parameters (including text preprocessors)	Cost: Medium Skills: No special skills required	Stakeholder: Product developer, system administrator, end user Tools: None Cost: Low
Generator	Cost: High Skills: Generative programming	Stakeholder: System administrator, end user Tools: Generator Cost: Low
Aspects	Cost: Medium Skills: Aspect-oriented programming	Stakeholder: Product developer Tools: Aspect-oriented language compiler Cost: Medium
Runtime conditionals	Cost: Medium Skills: No special skills required	Stakeholder: None Tools: None Cost: No development cost; some performance cost
Configurator	Cost: Medium Skills: No special skills required	Stakeholder: Product developer Tools: Configurator Cost: Low to medium

called the *variability guide* is reserved for exactly this purpose. The variability guide should describe each variation mechanism, how and when to exercise it, and what allowed variations it supports. The architecture documentation should also describe the architecture's instantiation process—that is, how its variation points are exercised. Also, if certain combinations of variations are disallowed, then the documentation needs to explain valid and invalid variation choices.

25.7 Evaluating a Product Line Architecture

Like any other, the architecture for a software product line should be evaluated for fitness of purpose. The architecture should be evaluated for its robustness and generality, to make sure it can serve as the basis for products in the product line's envisioned scope. It should also be evaluated to make sure it meets the specific behavioral and quality requirements of the product at hand. We begin by focusing on the what and how of the evaluation and then turn to when it should take place.

What and How to Evaluate. The evaluation will have to focus on the variation points to make sure they are appropriate, that they offer sufficient flexibility to cover the product line's intended scope, that they allow products to be built quickly, and that they do not impose unacceptable runtime performance costs. If your evaluation is scenario based, expect to elicit scenarios that involve instantiating the architecture to support different products in the family. Also, different products in the product line may have different quality attribute requirements, and the architecture will have to be evaluated for its ability to provide all required combinations. Here again, try to elicit scenarios that capture the quality attributes required of family members.

Often, some of the hardware and other performance-affecting factors for a product line architecture are unknown to begin with. In this case, evaluation can establish bounds on the performance that the architecture is able to achieve, assuming bounds on hardware and other variables. The evaluation can identify potential contention so that you can put in place the policies and strategies to resolve it.

When to Evaluate. An evaluation should be performed on an instance or variation of the architecture that will be used to build one or more products in the product line. The extent to which this is a separate, dedicated evaluation depends on the extent to which the product's requirements differ from the product line architecture envelope. If it does not differ, the product architecture evaluation can be abbreviated, because many of the issues normally raised in a single product evaluation will have been dealt with in the product line evaluation. In fact, just as the product architecture is a variation of the product line architecture, the product architecture evaluation is a variation of the product line architecture evaluation. Therefore, depending on the evaluation method used, the evaluation artifacts (scenarios, checklists, and so on)

will have reuse potential, and you should create them with that in mind. The results of evaluation of product architectures often provide useful feedback to the product line architects and fuel architectural improvements.

When a new product is proposed that falls outside the scope of the original product line (for which the architecture was presumably evaluated), the product line architecture can be reevaluated to see if it will suffice for it. If it does, the product line's scope can be expanded to include the new product, or to spawn a new product line. If it does not, the evaluation can determine how the architecture will have to be modified to accommodate the new product. The product line and product instance architectures can be evaluated not only to determine architectural risks but also to understand economic consequences (see Chapter 23), to determine which products will yield the most return.

25.8 Key Software Product Line Issues

It takes considerable maturity in the developing organization to successfully field a product line. Technology is not the only barrier to this; organization, process, and business issues are equally vital to master to fully reap the benefits of the software product line approach.

Architecture definition is an important activity for any project, but as we saw in the previous section, it needs to emphasize variation points in a software product line. Configuration management is also an important activity for any project, but it is more complex for a software product line because each product is the result of binding a large number of variations. The configuration management problem for product lines is to reproduce any version of any product delivered to any customer, where “product” means code and supporting artifacts ranging from requirement specs and test cases to user manuals and installation guides. This involves knowing what version of each core asset was used in a product's construction, how every asset was tailored, and what special-purpose code or documentation was added.

Examining every facet of launching a product line and institutionalizing a product line culture is outside the scope of this book, but the next sections will examine a few of the key areas that must be addressed. These are issues that an organization will have to face when considering whether to adopt a product line approach for software development and, if so, how to go about it.

Adoption Strategies

An organization's culture and context will dramatically affect how it goes about adopting a product line approach. Here are some of the important organizational and process factors that we have seen in practice.

Top-Down vs. Bottom-Up. Top-down adoption arises when a (typically high level) manager decrees that the organization will use the approach. The problem is to get employees in the trenches to change the way they work. Bottom-up adoption happens when designers and developers working at the product level realize that they are needlessly duplicating each other's work and begin to share resources and develop generic core assets. The problem is finding a manager willing to sponsor the work and spread the technique to other parts of the organization. Both approaches work; both are helped enormously by the presence of a strong *champion*—someone who has thoroughly internalized the product line vision and can share that compelling vision with others. (It works better if the champion is in a position of some authority.)

Proactive vs. Reactive. There are two primary models for how an organization may grow a product line:

- In a proactive product line, an organization defines the family using a comprehensive definition of scope. They do this not with a crystal ball but by taking advantage of their experience in the application area, their knowledge about the market and technology trends, and their good business sense. The proactive model allows the organization to make the most far-reaching strategic decisions. Explicitly scoping the product line allows you to look at areas that are underrepresented by products already in the marketplace, make small extensions to the product line, and move quickly to fill the gap. In short, proactive product line scope allows an organization to take charge of its own fate. Sometimes an organization does not have the ability to forecast the needs of the market with the certainty suggested by the proactive model. The proactive model also takes some time to define and implement, and in that time the organization needs to continue to construct products.
- In a reactive product line, an organization builds the next member or members of the product family from earlier products. This is best used when there is uncertainty of requirements. Perhaps the domain is a new one. Perhaps the market is in flux. Or perhaps the organization cannot afford to build a core asset base that will cover the entire scope all at once. In the reactive model, with each new product the architecture is extended as needed and the core asset base is built up from what has turned out to be common. The reactive model puts much less emphasis on up-front planning and strategic direction setting. Rather, the organization lets itself be taken where the market dictates. This is an example of agile architecting, as described in Chapter 15.

Incremental vs. Big Bang. If you are proactively building a product line, you still need to choose how to populate it: all at once or incrementally over time. Populating the core asset base all at once is a strategy that has worked successfully for some organizations. However, it tends to require all or nearly all of the

organization's resources be focused on that task, at the expense of new product production. A different approach is to populate the core asset base incrementally, as circumstances and resources permit. Each product that goes out the door is built with whatever core assets are available at the time. That means that early products will include software not derived from core assets. But those products will still be better off (that is, faster to market, of higher quality, and easier to maintain) than products built entirely from unique code. And it's entirely possible that some of the software unique to those early products can be extracted, adapted, and generalized to become core assets themselves, thus helping populate the core asset base in a reactive fashion.

Knowing the various adoption models can help an organization choose the one that is right for it. For example, the proactive model requires a heavier initial investment but less rework than the reactive model. The reactive model relies exclusively on rework with little initial investment. Which model should act as a guide for a particular organization depends on the business situation.

Creating Products and Evolving a Product Line

An organization that has a product line will have an architecture and a collection of elements associated with it. From time to time, the organization will create a new member of the product line that will have features both in common with and different from those of other members.

One problem associated with a product line is managing its evolution. As time passes, the line—or, more precisely, the set of core assets from which products are built—must evolve. That evolution will be driven by both external and internal sources:

External sources

- New versions of existing elements within the line will be released by their vendors, and future products will need to be constructed from them.
- New externally created elements may be added to the line. Thus, for example, functions that were previously performed by internally developed elements may now be performed by elements acquired externally, or vice versa. Or future products will need to take advantage of new technology, as embodied in externally developed elements.
- New features may be added to the product line to keep it responsive to user needs or competitive pressures.

Internal sources

- Some entity within the organization must determine if new functions added to a product are within the product line's scope. If so, they can simply be built from the asset base. If not, a decision must be made: either the enhanced product spins off from the product line, following its own evolutionary path, or the asset base must be expanded to include it. Updating the

line may be the wisest choice if the new functionality is likely to be used in future products, but this capability comes at the cost of the time necessary to update the core assets.

- An organization may wish to replace old products with ones built from the most up-to-date version of the asset base. Keeping products compatible with the product line takes time and effort. But not doing so may make future upgrades more time consuming, because either the product will need to be brought into compliance with the latest product line elements or it will not be able to take advantage of improvements in the line.

Organizational Structure

An asset base on which products depend, but which has its own evolutionary path (perhaps driven by technology change), requires an organization to decide how to manage both it and product development. There are two main organizational strategies from which to choose, plus a number of minor variations. The two main structures reflect different answers to the question “Shall we have a dedicated group whose sole job is to build and maintain our core asset base?”

1. *We’re all in this together.* In this scheme, there is no separate core asset group. The product-building development teams coordinate closely, and divide up the core asset responsibilities among themselves. That is, Product Team 1 might be assigned responsibility for the development and maintenance of Core Assets 3, 6, 9, 12, and 15; Product Team 2 might take Core Assets 1, 4, and 8; and so forth. This works well enough for small organizations, but as size grows the communication channels become untenable. Also, each team has to resist the temptation to build core assets that are especially appropriate to its needs, but less so to other teams’ needs.
2. *Separate core asset unit.* In this scheme, a special unit is given responsibility for the development and maintenance of the core asset base. Separate development teams in the organization’s business units build the products. In this scheme, the core asset unit (sometimes called a domain engineering unit) assumes the responsibility for the overall strategic direction of the product line. To the product teams, they appear almost like an external supplier. The product teams coordinate among themselves to set the core asset team’s development and test priorities, based on product delivery obligations.

25.9 Summary

This chapter presented an architecture-based development paradigm known as software product lines. The product line approach is steadily climbing in

popularity as more organizations see true order-of-magnitude improvements in cost, schedule, and quality from using it.

Like all technologies, however, this one holds some surprises for the unaware. Architecturally, the key is identifying and managing commonalities and variations, but nontechnical issues must be addressed as well, including how the organization adopts the model, structures itself, and maintains its external interfaces.

25.10 For Further Reading

[Clements 01a] is a comprehensive treatment of software product lines. It includes a number of case studies as well as a thorough discussion of product line “practice areas,” which are areas of expertise a product line organization should have (or should develop) to help bring about product line success.

[van der Linden 07] contains a rich set of product line case studies.

[Anastasopoulos 00] presents a good list of variation mechanisms, as do [Jacobson 97] and [Svahnberg 00]. [Bachmann 05] provides a list of their own, as well as a treatment of each in terms of cost (it was the source for Table 25.2). Organizational models for software product lines are treated in [Bosch 00].

There is an active software product line community of research and practice. The Software Product Line Conference (SPLC) is the mainstream forum for new software product line research and success stories. You can find it at www.splc.net. SPLC maintains a “Software Product Line Hall of Fame,” which showcases successful software product lines that can serve as engineering models (and inspiration) to aspiring product line organizations. Each year, new members of the Hall of Fame are nominated, and in most years a new candidate is inducted. You can see the winners at www.splc.net/fame.html.

The SEI’s website contains a wealth of material about software product lines, including a collection of “getting started” material: www.sei.cmu.edu/productlines.

25.11 Discussion Questions

1. Variability is achieved by adding variation mechanisms to a system. Variation mechanisms include inheritance, component substitution, plug-ins, templates, parameters (including text preprocessors), generators, aspects, runtime conditionals, and a configurator tool. Because variability can be seen as a kind of modifiability, see if you can map each of these variation mechanisms to one or more modifiability tactics given in Chapter 7.

2. Suppose a company builds two similar systems using a large set of common assets, including an architecture. Which of the following would you say constitutes a product line?
 - Sharing only an architecture but no elements.
 - Sharing only a single element.
 - Sharing the same operating system and programming language runtime libraries.
 - Sharing the same team of developers.

Defend your answer.

3. Pick a type of system you're familiar with—for example, an automobile or a smartphone. Think of three instances of that kind of system. Make a list of all of the things the three instances have in common. Now make a list of all of the things that distinguish the three instances from each other (that is, their variation points). If automobiles turn out to be too complex, start with a simpler kind of “system,” such as an electric light.
4. Write some concrete scenarios to express the variability you identified in the previous question.
5. Do the list of variation mechanisms in this chapter constitute tactics for variability? Discuss.
6. In many software product lines, products differ by the quality attributes they exhibit. For instance, a company might sell a cheap, low-security version of its product alongside a more expensive, high-security version of the same product. Which variation mechanisms might you choose to achieve this kind of variability?

This page intentionally left blank



PART FIVE

THE BRAVE NEW WORLD

Parts I through IV of this book have dealt with the technical, organizational, and business perspectives on software architecture. In this part, we turn our attention to emerging technologies. We have often been asked whether principles or technology is more important, and the answer, of course, is “Yes, they are both important.” Principles have a long lifetime; technology that affects architects tends to change every decade or so. In this part, we provide brief introductions to two technologies that we believe will last and have a significant impact on architects—the cloud and the edge. We also discuss one of the continuing problems of many architects: How do I get my organization to embrace architectural principles?

The cloud provides you with the option of outsourcing your data center. The vision is that computing resources are available to an application as electricity is available to a consumer. That is, one plugs in an appliance and electricity is available. In data center terms, you hook your web browser up to an application and computation power is available. All of the capacity, management, and operational issues of a data center are taken care of by a third party, and all you, as an architect, need to do is to utilize the resources you need. This trend has been accompanied by a vast expansion of the amount of data that organizations manage. Google, Yahoo!, Facebook, and the other web giants all must manage petabytes of data. In Chapter 26, we provide a brief introduction to the technologies associated with the cloud and with managing these vast amounts of data.

Cloud computing is associated with the world of social networks and open source. The term “edge-periphery” is used to describe both the crowdsource and the open source movements. The term refers simultaneously to crowdsourced

systems such as Facebook and Wikipedia and open source systems such as the Apache Web Server and Hadoop. In Chapter 27, we describe this phenomenon and explore some of the architectural implications of this aspect of the brave new world.

We end by discussing “adoption.” It describes an approach to dealing with the following problem: “OK, you guys have convinced me. Now I need to convince my organization of the importance of architectural principles. How do I do that?”